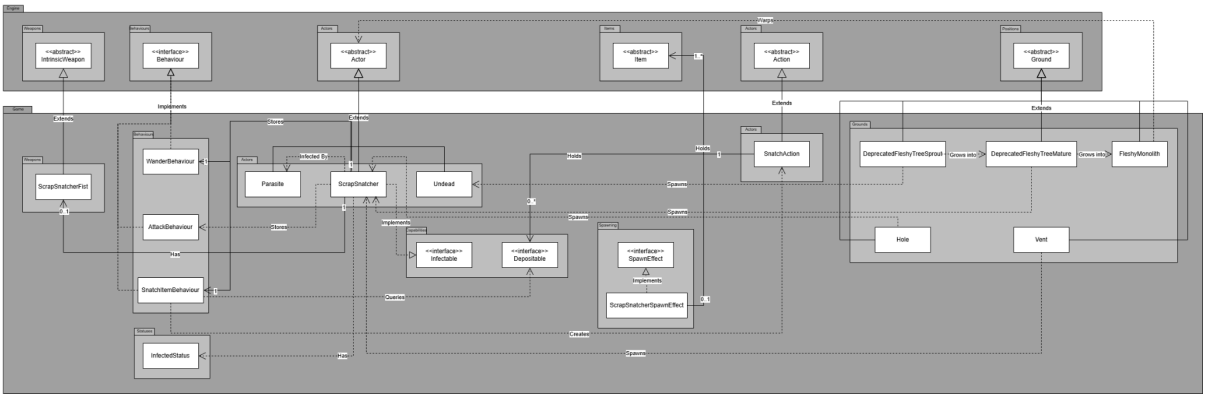
Design Diagram



Design Rationale - REQ 2

Switching Behaviours between Normal and Rabid

Design 1: Store behaviour maps and swap the behaviours performed depending on the rabid flag.

Pros	Cons
The logic for deciding which behaviour maps to use is separate from the behaviour classes themselves, allowing each behaviour to do one job (SRP).	Priority of behaviours is dependent on integers that the reader needs to learn (Connascence of Meaning)
The behaviours are able to be used across other creatures so no logic needs to be duplicated or rewritten (DRY).	

Design 2: Use conditional logic inside playTurn() to decide the action for the actor

Pros	Cons
Simple approach for the two actions that does not need extra classes (KISS)	Adding a new behaviour requires editing playturn() (violates OCP)
Everything is visible in one spot	Logic would not be able to be reused without duplicating (violates DRY)

Loot Explosion

Design 1: Utilise the existing SpawnEffectManager and add in the resource pool when registering the effect

Pros	Cons
The spawners and the effects do not need to reference each other and instead depend on the manager abstraction (DIP)	
Resource pool is added in one place so	

adding a new resource does not need to duplicate any dropping logic (DRY)	
New effects can be registered without modifying spawners (Open-Closed Principle)	

Design 2: Hardcode the dropping logic inside each spawner

Pros	Cons
All of the behaviour sits inside the spawner and is easy to read (KISS)	Explosion logic is rewritten for every spawner (violates DRY)
	Each spawner now has the responsibility of spawning and scattering loot (violates SRP)

Deprecated Fleshy Tree Implementation

Design 1: Have each stage as a separate Ground that replaces itself with setGround() when growing

Pros	Cons
Each stage considers only its own spawning and growth (SRP)	Stage must name the next stage's class when growing (Connascence of Name)
Deprecated variants were added without editing the normal tree classes (Open-Closed Principle)	

Design 2: A single DeprecatedFleshyTree class that has conditionals to determine behaviour and growth

Pros	Cons
Everything occurs in one class with less types and subclasses (KISS)	Many responsibilities for each stage (violates SRP)

Final Design

The final design uses all of the Design 1's listed above. All together, they allow for the Scrap Snatcher to be easily extended and stop the classes from depending too much on each other.

For the behaviour logic, we had the Snatcher store two behaviour maps and then run the one that matches the rabid flag. If the Snatcher was infected and the rabid flag was ticked to true, the rabidBehaviours would be actioned and vice versa for the non-infected case. Having the behaviours stored separately and as their own classes allows for them to be reused by other creatures without needing to be rewritten (DRY). The only logic that needs to be handled by the creature is the prioritisation of behaviours. However, the downside of this is that the priorities rely on integers that the user needs to learn and understand (Connascence of Meaning). This is still a better alternative however to filling playTurn() inside the Snatcher with conditionals. A new behaviour is able to be added just by simply populating the existing maps (Open-Closed Principle).

The loot explosion reuses the existing SpawnEffectManager and then passes the resource pool in when the effect is registered. This means that the spawners themselves and the effects that occur do not have to reference each other and only need to reference the manager (DIP). The list of droppable resources is also written in one place, the EclipseNebula, when it is registered without needing to be repeated for every spawner (DRY). New resources and new effects can also be added in this way without modifying any of the spawners (Open-Closed Principle) as opposed to hardcoding the drop logic into spawners and then duplicating it.

DeprecatedFleshyTree's stages are their own grounds and swap with each other with setGround() when growth occurs. Each stage only needs to worry about their own spawning and growth logic (SRP) and the deprecated versions were added without editing the original FleshyTree classes (Open-Closed Principle). This is much cleaner than having the behaviours and logic placed into a single class with many conditionals.